SYSTEM ARCHITECTURE & QUALITY AUDIT REPORT

รายงานการตรวจสอบสถาปัตยกรรมระบบและข้อเสนอแนะเชิงลึก (Audit & Strategic Recommendations)

แอปพลิเคชัน AI Agent Visual Workflow Builder & Document Analytics Platform (LGv3)

8.6/10	100%	7.2/10	7.8/10
สถาปัตยกรรมรวม	BACKEND TESTS (45/45)	FRONTEND ARCHITECTURE	SECURITY & GOVERNANCE

ชื่อระบบ / โครงการ:	AI Agent Visual Workflow Builder (LGv3)
Production Endpoint:	`https://aibuilder-dev.addpay.in.th`
Backend Engine:	FastAPI 0.115 • LangGraph 0.2.20 • SQLAlchemy 2.0 Async
Frontend Stack:	React 19.2 • Vite 8.0 • Tailwind CSS v4 • React Flow
In-Process Analytics:	Polars (Parquet) • DuckDB High-Speed In-Memory SQL
Security Framework:	OAuth2 Password Bearer • JWT Token • 5-Tier RBAC
วันที่จัดทำรายงาน:	2 พฤศจิกายน 2026 (ระบบ Release v3.0)

1. บทสรุปสำหรับผู้บริหาร (Executive Summary)

แอปพลิเคชัน **AI Agent Visual Workflow Builder (LGv3)** เป็นแพลตฟอร์ม Low-Code/No-Code สำหรับออกแบบ ควบคุม และประมวลผลกระบวนการทำงานอัตโนมัติด้วย Multi-Agent AI (รองรับ Google Gemini, OpenAI, Anthropic, DeepSeek, Local LLMs) ควบคู่กับการสกัดและวิเคราะห์เอกสารขนาดใหญ่ (PDF, Excel, Parquet) จากการตรวจสอบซอร์สโค้ดและระบบประมวลผลทั้ง Frontend และ Backend อย่างละเอียด พบว่าระบบมีรากฐานสถาปัตยกรรมที่แข็งแกร่ง มีความพร้อมในการใช้งานระดับองค์กร (Enterprise-Ready) สูงมาก แต่ยังมีจุดเสี่ยงเชิงสถาปัตยกรรม (Architectural Risks) และจุดที่ควรเพิ่มประสิทธิภาพเพื่อรองรับการขยายตัว (Scalability) ในอนาคต

จุดเด่นของระบบ (Core Strengths) EXCELLENT

- **LangGraph Engine:** แปลง Visual Canvas เป็น StateGraph แบบ Async สมบูรณ์แบบ รองรับการแยกเงื่อนไข (Branching) และ Subflow Caching
- **DuckDB & Polars Engine:** สกัดและคิวรีไฟล์ข้อมูลระดับล้านเรคคอร์ดในแรมโดยตรง รวดเร็วโดยไม่ต้องพึ่งพา Data Warehouse ภายนอก
- **Enterprise RBAC & Isolation:** ควบคุมสิทธิ์ 5 ระดับ แยกความเป็นเจ้าของ Workflow ชัดเจน และมีระบบ Sharing (View/Edit/Execute)
- **Backend Test Coverage:** ผ่านการทดสอบ Automated Test ครบถ้วน **45/45 Test Cases (100% Pass)**

ประเด็นเร่งด่วนที่ควรปรับปรุง (Key Areas to Improve) ATTENTION

- **Monolithic App.tsx:** มีโค้ด 1,848 บรรทัด พร้อม 40 `useState` ทำให้เกิดปัญหา Re-render และยากต่อการทดสอบ/บำรุงรักษา
- **ขาด State Management ส่วนกลาง:** ไม่มี Zustand/Redux ใช้การส่ง Props ลึกหลายชั้น (Prop Drilling)
- **Frontend ขาด Automated Test:** ยังไม่มี Vitest หรือ React Testing Library เพื่อทดสอบส่วน UI components
- **CORS & Fallback JWT Secret:** ควรบังคับดึงค่า Secret จาก Environment Variable เท่านั้น และจำกัด CORS Origins ให้รัดกุม

2. การตรวจสอบสถาปัตยกรรมระบบ Backend & Execution Engine

ระบบ Backend พัฒนาด้วย **FastAPI 0.115** บน Python 3.11/3.12 มีการจัดโครงสร้าง Layered Architecture ชัดเจน ประกอบด้วย Routers, Services, Schemas, Models และ Graph Engine:

โมดูล / องค์ประกอบ	เทคโนโลยีที่ใช้งาน	ผลการตรวจสอบและการประเมินคุณภาพ
Graph Engine	`langgraph`, `langchain-core`	ยอดเยี่ยม แปลง Node/Edge เป็น StateGraph รองรับ State Streaming (SSE), Node Skip, และ Resume Cache
Database & ORM	SQLAlchemy 2.0 (Async), aiosqlite / asyncpg	ดีมาก รองรับทั้ง SQLite และ PostgreSQL มี Connection Pooling และ Migration Schema ที่สมบูรณ์
Big Data & Analytics	`polars 1.6`, `duckdb 1.0`	ยอดเยี่ยม สามารถแปลง CSV/Excel เป็น Parquet และรัน In-Memory SQL บนไฟล์ข้อมูลขนาดใหญ่ได้เสี้ยววินาที
Task Automation	`APScheduler 3.10`	ดีมาก ตั้งตารางรัน Workflow เบื้องหลังแบบ Cron syntax และ Interval ได้แม่นยำ

โมดูล / องค์ประกอบ	เทคโนโลยีที่ใช้งาน	ผลการตรวจสอบและการประเมินคุณภาพ
Execution Isolation	In-Process Async Tasks	ข้อสังเกต รันงานอยู่ภายใน Event Loop ของ FastAPI หากมีงาน LLM หนักมากๆ อาจแย่ง Thread ทรัพยากรของ Web Server

ผลการทดสอบ Backend Automated Test Suite: ผ่านการรัน Pytest 45 ชุดทดสอบ (`test_analytics.py` , `test_auth.py` , `test_langgraph_engine.py` , `test_project_groups.py` , `test_rbac.py` , `test_user_queues.py` , `test_workflow_rbac.py` , `test_workflow_sharing.py` , `test_workflows.py`) ใช้เวลา 134.2 วินาที ผ่านทั้งหมด 100%

3. การตรวจสอบสถาปัตยกรรม Frontend & Code Quality

Frontend พัฒนาด้วย **React 19.2**, **Vite 8.0**, **Tailwind CSS v4** และ **React Flow (@xyflow/react)** โครงสร้างภาพรวมทันสมัย มีการแยกหน้าย่อยชัดเจน แต่ตรวจพบประเด็นความซับซ้อนเชิงเทคนิคที่ควรได้รับการปรับปรุง:

App.tsx Complexity REFACTOR

1,848 บรรทัด ในไฟล์เดียว

- `useState` : **40 ตัว**
- `useEffect` : **15 ตัว**
- `useRef` : **14 ตัว**
- `useMemo` : **0 ตัว**

Bundle Size & Chunks OPTIMIZE

Total JS: 1.89 MB

- `index.js` : 1.89 MB (511 KB gzip)
- `pdf.worker.mjs` : 1.05 MB
- `index.css` : 169 KB (24 KB gzip)
- ไม่มี Code-Splitting แยก Route

Test & Verification MISSING

0% Automated Coverage

- ไม่มี Vitest หรือ Jest
- ไม่มี React Testing Library
- ไม่มี E2E Playwright/Cypress
- พึ่งพา Manual Test เพียงอย่างเดียว

การวิเคราะห์เชิงลึกประเด็น Frontend:

1. **การจัดเก็บ State ที่กระจัดกระจายและขาด Centralized Store:** ปัจจุบัน State ของ Canvas, Workflow Nodes/Edges, History Undo/Redo, Modal Dialogs, และ Search Filters ถูกประกาศกองรวมอยู่ใน `App.tsx` แล้วส่งต่อผ่าน Props ลงไปกว่า 10 ลำดับ (Prop Drilling) ทำให้เมื่อ State ใดตัวหนึ่งเปลี่ยน คอมโพเนนต์ที่ไม่เกี่ยวข้องทั้งหมดจะถูก Re-render ซ้ำโดยไม่จำเป็น
2. **การขาด Dynamic Code-Splitting (Lazy Loading):** Vite ทำการ Build โค้ดทั้งหมดทั้ง React Flow, Lucide Icons, Tiptap Editor, PDF Viewer และ Settings Pages รวมเป็นก้อนเดียว `index.js` ขนาด 1.89 MB ส่งผลให้ผู้ใช้ที่เข้าหน้า Dashboard ต้องดาวน์โหลดโค้ด Canvas และ Editor ขนาดใหญ่ตั้งแต่วินาทีแรก
3. **การจัดการ Routing ด้วย Manual History API:** ใน `App.tsx` ใช้ `window.history.pushState` และฟังอีเวนต์ `popstate` โดยตรง แทนที่จะใช้ Declarative Router ที่เป็นมาตรฐาน (เช่น `react-router-dom` หรือ `TanStack Router`) ซึ่งอาจทำให้การจัดการ Nested Routes, Query Parameters, และ Route Guards มีความเปราะบาง
4. **การขาด Global Error Boundary:** หากเกิดข้อผิดพลาดในการ Render ของ Custom Node หรือ Third-party Library ใน Canvas หน้าจอจะขึ้นสีขาว (White Screen of Death) ทั้งระบบทันที ควรครอบด้วย React Error Boundary พร้อมหน้า UI สำรอง (Fallback UI)

4. การตรวจสอบความมั่นคงปลอดภัยและการจัดการสิทธิ์ (Security Review)

หัวข้อตรวจสอบ	ระดับความเสี่ยง	รายละเอียดการตรวจสอบและแนวทางแก้ไข
JWT Secret Key Fallback	ความเสี่ยงสูง	ใน `config.py` มีการกำหนดค่า Default Hardcoded Key (`ai-agent-project-lgv3-jwt-secret-key ...`) หาก Production ไม่ตั้งค่า `JWT_SECRET_KEY` ใน .env ผู้โจมตีอาจปลอมแปลง Token ได้ ควรตั้ง Exception ให้ Server หยุดทำงานทันทีหากไม่พบ Key ใน Environment
CORS Origins Wildcard	ปานกลาง	ใน `main.py` มีการตั้ง `allow_origins=["*"]` พร้อมกับ `allow_credentials=True` ซึ่งอาจเสี่ยงต่อการถูก Cross-Site Scripting หรือส่งคำขอข้ามโดเมนที่ไม่ได้รับอนุญาต ควรระบุเฉพาะโดเมนทางการ เช่น `https://aibuilder-dev.addpay.in.th`
Database Multi-User Concurrency	ปานกลาง	ใน `docker-compose.yml` ค่า Default `DATABASE_URL` ยังชี้ไปที่ SQLite แม้จะมี PostgreSQL Service รันอยู่ หากมีผู้ใช้เขียนข้อมูลพร้อมกันจำนวนมาก

หัวข้อตรวจสอบ	ระดับความเสี่ยง	รายละเอียดการตรวจสอบและแนวทางแก้ไข
		SQLite อาจเกิดปัญหา `database is locked` ควรสลับไปใช้ PostgreSQL แบบสมบูรณ์
Password Hashing (Bcrypt)	ปลอดภัยสูง	ใช้ Passlib + Bcrypt อัลกอริทึมมาตรฐาน ปลอดภัย ไม่มีการจัดเก็บรหัสผ่าน Plaintext ในฐานข้อมูล
Granular RBAC Engine	ปลอดภัยสูง	แบ่งสิทธิ์ผู้ใช้ 5 ระดับชัดเจน พร้อมตรวจสิทธิ์ในทุก Endpoint ของ Workflows, Users และ Project Groups

5. ข้อเสนอแนะเชิงกลยุทธ์และแนวทางพัฒนา (Actionable Recommendations)

เพื่อให้ระบบมีความเสถียร มีประสิทธิภาพสูงสุด รองรับผู้ใช้งานระดับองค์กรได้นับหมื่นราย และพร้อมสำหรับการต่อยอดระยะยาว ขอเสนอแนวทางการพัฒนาแบ่งตามระดับความสำคัญดังนี้:

ระยะสั้น: Quick Wins & Security Hardening (1-2 สัปดาห์)

PRIORITY 1

- **บังคับความปลอดภัย JWT Secret:** ลบ Default Hardcoded Secret ใน `config.py` และเขียนเงื่อนไขตรวจสอบให้แอปหยุดทำงานหากไม่พบ Secret Key ใน Production
- **จำกัด CORS Origins:** ปรับ `allow_origins` ให้รับเฉพาะโดเมนของระบบและ Localhost เท่านั้น ปิดการใช้ `*` คู่กับ Credentials
- **เปิดใช้งาน PostgreSQL บน Production:** สลับคอนฟิกใน Coolify / Docker Compose จาก SQLite มาใช้ PostgreSQL Service ที่สร้างไว้ เพื่อรองรับ Concurrent Transactions
- **เพิ่ม Rate Limiting:** ติดตั้ง `slowapi` หรือ Redis Rate Limiter บน Endpoint `/api/auth/token` และ AI Execution เพื่อป้องกัน Brute-Force และ API Abuse

ระยะกลาง: Frontend Refactoring & Performance (3-4 สัปดาห์)

PRIORITY 2

- **ย้าย State สู่ Zustand Store:** สกัด 40 `useState` ใน `App.tsx` ออกเป็น 3 โมดูลย่อย (`useWorkflowStore` , `useCanvasStore` , `useUiModalStore`) เพื่อลด Re-render และลดขนาด App.tsx เหลือไม่เกิน 300 บรรทัด
- **ทำ Code-Splitting ด้วย React.lazy:** โหลดเฉพาะหน้าที่ผู้ใช้งานเปิด เช่น แยก `CanvasFlowArea` , `SettingsPages` , และ `Tiptap` ลดขนาด Initial Chunk ลงเหลือ < 450 KB
- **ติดตั้ง Global Error Boundary:** ป้องกันหน้าขาวเมื่อเกิด Error ใน Custom Node หรือ Canvas rendering
- **ติดตั้ง Vitest & React Testing Library:** เขียน Unit Test สำหรับ Custom Hooks และ Node Components

ระยะยาว: Architecture Scaling & Enterprise Features (1-3 เดือน)

PRIORITY 3

- **Distributed Task Queue (Celery / Redis / ARQ):** แยก Worker สำหรับรัน LangGraph ออกจาก FastAPI Main Process เพื่อให้ระบบรองรับ Long-Running Document Processing หลายร้อยคิวพร้อมกัน
- **CI/CD Automated Quality Gates:** เพิ่ม GitHub Actions ให้รัน Lint, TypeScript Build Check, และ Pytest อัตโนมัติทุกครั้งก่อน Pull Request หรือ Deploy ขึ้น Coolify
- **Real-time Multi-User Collaboration:** ประยุกต์ใช้ Yjs หรือ WebSockets เพื่อให้ผู้ใช้งานในทีมสามารถเปิดแก้ไข Canvas เดียวกันได้พร้อมกันแบบ Real-time
- **Prompt & Model Cost Observability:** ติดตั้ง OpenTelemetry / Langfuse / Helicone เพื่อติดตามค่าใช้จ่าย Token และ Latency ของ AI แต่ละโหนดอย่างละเอียด

สรุปภาพรวมและแผนภูมิสถานะระบบ (Status Summary)

SUMMARY

ความพร้อมในการให้บริการ (Readiness): **พร้อมใช้งานจริงระดับ Production (Production Ready)**

ประสิทธิภาพ Engine: รวดเร็วมากด้วย Polars + DuckDB และ LangGraph

การปรับปรุงที่ผ่านมา: แยกหน้า Settings ชิดซ้าย, ระบบ Role-Based Isolation สมบูรณ์, และ LiteLLM Menu ตอบโจทย์ UI/UX ทันสมัย

เป้าหมายถัดไป: ปรับโครงสร้าง State Frontend และเพิ่ม Automated Test Suite

จัดทำและตรวจสอบโดย: ทีมวิศวกรรมระบบและสถาปัตยกรรมซอฟต์แวร์ AI Agent Visual Platform

เอกสารฉบับนี้เป็นรายงานการวิเคราะห์และข้อเสนอแนะเชิงเทคนิค เพื่อการพัฒนาและยกระดับแอปพลิเคชันอย่างยั่งยืน